Abraham Horta

CSC-499-01

Capstone Final Report

Introduction

Today, modern transaction systems, specifically financial systems, heavily rely on APIs to process transactions, and to take care of sensitive user/customer information. While many APIs support improved efficiency and scalability, they may also introduce major risks, regarding security, when/if they are not designed properly. In addition, according to the OWASP top 10, common vulnerabilities such as SQL injection, weak password storage, broken authentication, insecure session handling, and improper authorizations can lead to fraudulent transactions(in this case financial fraud), unauthorized access, and the exposure and/or loss of sensitive information[1].

As mentioned in the original project proposal, This capstone project will focus on the design, implementation, and evaluation aspects of a secure, transaction-based backend system. To support, the system will implement secure financial transactions as the transaction type. The system will be created by using an API-first architecture, backed by a relational database and supported by a light frontend interface that will mainly be used for the sole-purpose of demonstrating system workflows and access control boundaries.

The main goal of the project is to demonstrate real-world backend engineering and security practices, which include database design, authentication and authorization, secure API development, and performance considerations. For this project, the implementation of the secure financial transaction API system will be using FastAPI,

MySQL, and Python, connected with the lightweight frontend interface. To emphasize, the main objective of this project was not to simply create a CRUD-based application, but to build a security-first system that demonstrates secure authentication, protected transaction handling, the enforcement of database integrity, in addition to security testing.

The project focuses on the processing of financial transactions, because financial systems need to have strong guarantees of confidentiality, integrity, and accountability. The users of the system need to be authenticated properly, the transactions must adhere to strict business rules, and for the purpose of future auditing, all important actions need to be logged. These requirements make financial systems, such as this project, an ideal and relevant use case for demonstrating secure principles regarding software engineering.

The final implementation of the project includes user registrations, secure logins with the use of password hashing, JWT-based authentication, the support of role-based access control(RBAC), the creation of transactions, transaction status updates, rollback protection, audit logging, frontend integration, and an extensive security testing phase, including manual and automated testing, using Kali Linux and Burp Suite tools.

Project Description and Definition

Problem Statement:

Many web applications fail because their main focus is on functionality, whereas they ignore critical aspects such as security architecture. A basic transaction system might

allow users to log in and make transactions, but without the proper controls, it can easily open up/introduce numerous vulnerabilities, causing it to be susceptible to attacks like:

- SQL injection through login forms
- Unauthorized access to protected endpoints
- Password theft because of plaintext storage
- Tampering with transactions through request manipulations
- Invalid transaction state changes
- Missing audit trails possibly linked to suspicious activity, etc.

These notable weaknesses are dangerous, especially in financial systems, because they involve the movement of money, sensitive customer information/data, and legal responsibilities and accountabilities.

Why This Problem is Important:

Financial applications arguably need stronger protection than standard CRUD applications, because they have a direct impact on business operations and the trust of customers/users. If an attacker is able to bypass authentication, manipulate/alter transactions, or hide malicious activity, the results can be detrimental. The consequences can include fraud, data breaches, and regulatory violations.

In addition, this project addresses these issues by enforcing security measures at both the application layer, and the database layer.

Project Goals:

The system was designed for achieving the following objectives:

- Register and Authenticate users securely
- Prevent SQL injection attacks
- Protect sensitive endpoints by using token-based authentication
- Enforce valid transitions between the state of transactions(e.g. “Pending” to “Completed” to “Refunded”))
- Maintain proper rollback protection for operations that fail
- Record audit logs for accountability
- Provide a usable frontend for testing and demonstration
- Verify/validate security controls through security/penetration testing methods/techniques

Implementation

Hardware:

The project was developed using a personal laptop system with resources for local API development, MySQL database execution, frontend rendering, and security testing.

- Hardware included:
  - Intel-based processor
  - Minimum 16GB RAM
  - Local SSD storage
  - Virtual machine support for Kali Linux

- Stable internet connection for documentation access and dependency installation

Because the system was designed for local development and security testing rather than enterprise deployment, no cloud infrastructure was required during implementation.

Operating System(s):

The operating systems that were used for this project include Windows 10/11, and a Kali Linux Virtual Machine(Debian-based) operating system. This project began on a windows 10 machine, however, the system began to give me difficulties. In order to conduct proper security testing and continue project completion, I had to upgrade to a new laptop, which also came with a Windows 11 update. Additionally, all of the software was reinstalled and the project resumed. Windows acted as the main development environment for coding, configuring the database, FastAPI setup, frontend implementation, and server execution through Pycharm and Powershell.

In addition, the Kali Linux VM was used for the sole purpose of penetration testing and security validation due to its suite of tools, which were necessary for testing SQL injection protection, authorization controls, and transaction tampering prevention. More details will be provided in the experimental results section of the report.

Using both operating systems allowed the project to simulate the side/perspective of the developer and the side/perspective of the attacker, which helped to make a more realistic and complete security/system “hardening” process.

Programming Languages:

Python 3.14 was chosen and used as the main programming language for backend development over other common backend options, such as Node.js and Go, for multiple reasons. Based on conducting previous research, Python was chosen because of its great balance and combination of productivity, versatility, performance, scalability, and ease-of-use[2]. In addition, python contains extensive libraries that can all work together, which is very useful, and it supports and is compatible with popular technologies and backend frameworks which include Django, Flask, FastAPI, and more[2]. It enabled the development speed to increase, while supporting all of the project/system's functions.

SQL was also used alongside Python for database implementation, supporting schema creation, stored procedures, role validation, transaction state enforcement, and audit logging.

HTML, CSS, and JavaScript were also used for the lightweight frontend/interface. HTML helped with the page structure, the styling and presentation of the page was handled by CSS, and JavaScript was used to communicate with the backend API endpoints, also supporting the processing of the login and transaction requests from the browser.

Together, this full-stack combination helped to create secure backend functionality along with a usable browser-based frontend.

Development Tools and Security Testing Software :

Several tools were used for development and testing throughout implementation.

Pycharm

- Used as the main IDE for Python code and project organization.

MySQL Workbench

- Used as a specialized IDE for developing and writing SQL for creating the database schema, managing stored procedures, and query testing. In addition, it allowed the database elements to be visualized graphically, which included being able to view tables, columns, and provided the ability to reverse engineer and create an entity relationship diagram to view the overall layout and design decisions of the database/schema.

Uvicorn

- Used as the ASGI server to locally run the FastAPI app throughout the entire project.

Kali Linux/Kali Linux VM

- Used specifically for penetration testing and severity validation/verification.

Burp Suite

- Used for automated security tests which included manual interception and modification of HTTP requests in order to test authorization bypass attempts, parameter tampering, etc.

SQLMap

- Used for automated SQL injection testing .

Curl

- Used for directly testing protected API endpoints with the command line.

Software Packages and Libraries:

Multiple software packages and libraries were used to help support the implementation of the system. Each package was used to serve a different purpose in backend development, authentication, frontend rendering, database connectivity, and security testing.

FastAPI

- FastAPI was used as the primary framework for building the REST API. It handled the project's route definitions, request processing, validation, dependency injection, JWT authentication, and API execution. FastAPI was chosen because it is a modern/industry approach that provides high performance, higher development speeds, easy to use and get familiar with, it has strong support for API development, and it also contains very useful and automatic interactive documentation with OpenAPI[3].
    - Key imports included:
        - "from fastapi import fastapi"
            - This import was used to create the main FastAPI application object. It acts as the core of the backend system, and

handles routing, endpoint definitions, request handling, and API execution[4].

- “from fastapi import depends”
    - This import was used for dependency injection in FastAPI, mainly for protected routes that required authentication. Also, it allowed secure endpoints to automatically verify JWT tokens before permitting access[4].
- “from fastapi import HTTPException”
    - This import was used to return proper API error responses like 401 Unauthorized, 404 Not Found, or 409 Conflict. It also made error handling structured and professional instead of returning generic failures[4].
        - To add extra context, the appropriate status code for a username that already exists is the HTTP status code 409 Conflict, and many systems include the detail “Username or Email already exists”. However, I personally and purposely changed the registration HTTP status code to 400 and the detail to “Registration Failed”. I did this adjustment because it limits any information that may be useful to a person with malicious intent. For example, if a malicious person can see if a username or email already exists, it can confirm if a certain individual is using the

platform, therefore the malicious actor can begin conducting targeted attacks, building a profile, etc.

- “from fastapi import Request”
  - This import was needed when rendering Jinja2 templates. It enabled FastAPI to pass request information into HTML templates so pages can be rendered correctly[4].
- “from fastapi.responses import HTMLResponse”
  - This import was utilized to return HTML content from FastAPI routes instead of JSON responses. It also made the browser-based frontend possible for the overall project[4].

FastAPI Security

- FastAPI Security features were used to incorporate Bearer Token authentication for protected routes, making sure that only authenticated users who obtain valid JWT tokens can create or modify financial transactions.
  - Key imports included:
    - “from fastapi.security import HTTPBearer”
      - This import package was used for implementing Bearer Token authentication, allowing the API to safely obtain JWT tokens from the Authorization header at the time of protected requests[5].
    - “from fastapi.security import HTTPAuthorizationCredentials”
      - This import worked alongside HTTPBearer by extracting the actual token credentials from incoming requests. It also

enabled the backend to read and verify/validate the JWT that was provided by the user[5].

Pydantic

- “Pydantic is the most widely used data validation library for Python.”[6] It was chosen because, according to the Pydantic Docs, it makes data validation easier, quicker, and more dependable by using Python type hints, mitigating errors while improving the quality of code,and improving integration with modern development tools[6].
    - Key imports included:
        - “from pydantic import BaseModel”
            - For the project, Pydantic was used for request validation and data modeling. Because the project’s Python classes are inheriting from BaseModel, BaseModel allows structured request bodies like login requests, registration requests, and transaction requests to be automatically validated before being processed by the backend[6]. This helped to improve security by preventing incorrect/invalid input from reaching business logic.

MySQL Connector(mysql-connector-python)

- MySQL Connector is a standard database connector/driver that is used for a web framework, FastAPI, to make a direct connection to a MySQL database, in this case using Python[7].

- Key Imports Included:
  - “import mysql.connector”
    - For the project, this package was used to connect the FastAPI backend to the MySQL database, allowing Python to execute SQL queries, call Stored Procedures, fetch results, and manage database transactions like commits and rollbacks.

python-jose(JWT)

- The python-jose library was used to generate and confirm JSON Web Tokens(JWTs). After logins were successful, the system would generate access tokens that contained user id information and expiration timestamps, which were also required for the financial operations.
  - Key Imports Included:
    - “from jose import jwt”
      - This library was used for generating and decoding JWTs, handling secure token creation for authenticated users and token verification when protected endpoints were accessed[8].
    - “from jose import JWTError”
      - This import was used for exception handling during token verification, so if a token is invalid, expired, or tampered with, JWTError triggered secure rejection of the request[8].

Passlib and bcrypt(passlib[bcrypt])

- Passlib with bcrypt was used for the password hashing and password verification.
    - Key Imports included:
        - “from passlib.context import CryptContext”
            - This package was used for secure password hashing with bcrypt, so instead of storing passwords in the database as plaintext, it enabled the passwords to be encrypted before they were stored[9].

Jinja2 Templates

- Jinja2 templates were chosen for this project to support the lightweight frontend design without the need for a full React or Angular framework.
    - Key Imports included:
        - “from fastapi.templating import Jinja2Templates”
            - This import was used for frontend rendering using HTML templates, allowing FastAPI to serve HTML pages while integrating backend data into the frontend interface[10][11].

Static Files

- Static file handling helped to improve the code organization by separating the frontend from the backend logic.
    - Key Imports included:

        - “from fastapi.staticfiles import staticfiles”
            - This package was used in the project to serve static frontend resources like the .CSS file and the .js file apart from the backend code[12][13].

Python Datetime

- “The datetime module supplies classes for manipulating dates and time.”[14] “While date and time arithmetic is supported, the focus of the implementation is on efficient attribute extraction for output formatting and manipulation.”[14]
    - Key Imports included:
        - “from datetime import datetime”
            - This import was used to generate timestamps and deal with the expiration times for the JWT tokens,[14].
        - “from datetime import timedelta”
            - This import was/is used with datetime to specify how long JWT tokens remain active[14].

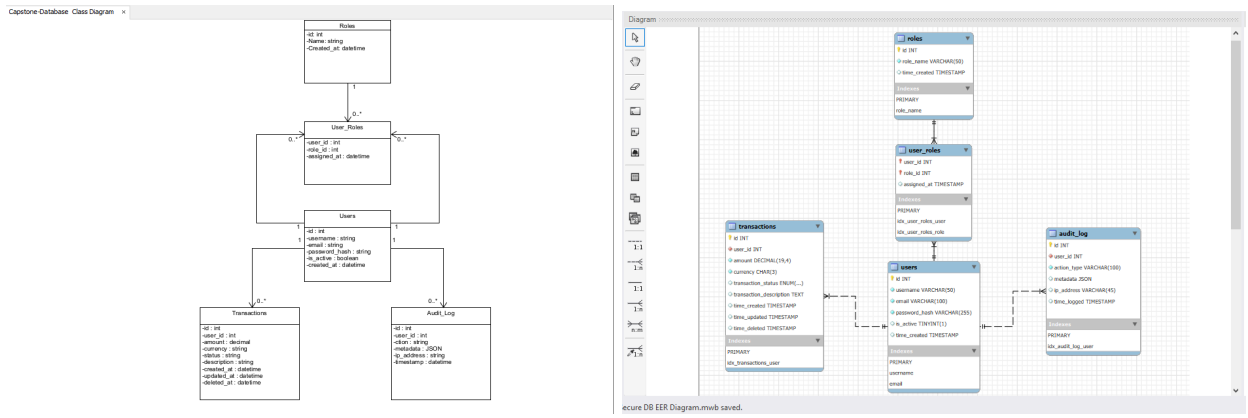
System Architecture:

- The project follows a client-server architecture where users interact with the system through browser-based forms on the frontend for registering, logging in, and managing transactions/the status of transactions if they are authorized. To support what was previously mentioned, the FastAPI backend is responsible for handling requests, validating JWT authentication, verifying user roles, applying business logic, and connecting to MySQL stored procedures for executing secure

transactions. MySQL stores/records all system data, which includes users, roles, transactions, and audit logs, while the stored procedures and rollback protection help to maintain database integrity, supporting the maintainability, scalability, and security.

Database Design:

Logical Design vs. Physical Schema Confirmation



The UML diagram(left) represents the original logical data model, defining attributes, intended relationships, and entities. The reverse engineered EER diagram(right) generated in MySQL workbench, verifies that the implemented database accurately reflects the pre-planned architecture. The five core tables(**detailed below**) were successfully created and connected, and the physical schema maintains relationship cardinality and normalization standards, and applies foreign key constraints.

Key Architectural Decisions

- RBAC implementation: Many-to-many through the user-roles allows for expandable role assignment separation of authentication and authorization.
- Financial Precision: Transactions use Decimal(19,4) for higher accuracy and prevents floating-point rounding errors.
- Soft Deletes: transactions.time_deleted allows for deletion, while saving historical data in regards to audit/compliance purposes.
- Audit Logging: The audit_log table supports structured event tracking, along with metadata storage for traceability.

- Referential Integrity: All foreign keys are enforced and indexed for optimized query performance and relational consistency.

The database contains the following major tables that support critical elements such as authentication, authorization, transactions, and auditing:

- **users**
    - This table stores user account information including username, email, and password hashes/ciphertext using bcrypt hashing.
- **roles**
    - This table stores predefined system roles, which include Admin(ID = 1), Support(ID = 2), and Customer(ID = 3). These roles are used to determine which operations are able to be performed by the user.
- **user_roles**
    - This table maps the users to assigned roles, allowing role-based access control(RBAC) by connecting/linking users to different authorization levels(unique IDs included above).
- **transactions**
    - This table is used to store the transaction(financial transaction) details, which include the transaction amount, currency type, description, owner, and the current transaction status such as “Pending”, “Completed”, “Refunded”, or “Failed”.
- **audit_log**
    - This table is used to record sensitive operations, such as transaction creation, status updates, and administrative actions. It provides

accountability, traceability, and can help support security investigations if needed.

Stored Procedure Implementation:

Stored procedures were implemented instead of direct SQL updates, because allow for more control over financial operations and help to improve backend security, if done properly[15].

- Reasons for implementing include[15]:
    - Centralized business rules
    - Reduced duplicate validation logic
    - Better rollback handling
    - Improved consistency across all API calls
    - Protection against accidental invalid updates
- For example, a transaction that has a recorded status of “Completed”, should not be allowed to return to “Pending”. Rather than fully depending on the frontend for validation, the database enforces this rule. This also ensures security even if someone was to bypass the frontend using curl, BurpSuite, etc.

SQL Injection Prevention Implementation:

For this project, parameterized queries were used instead of string concatenation in order to prevent SQL injection attacks. Building SQL with the use of string concatenation enables attackers to inject malicious SQL commands into the query[15]. In order to mitigate/combat this, the project uses placeholders, %, which help to make sure that user input is treated as data instead of SQL logic that can be executed[15]. Additionally, if someone were to enter malicious input such as ‘OR 1=1 -- , the database will treat it as literal text instead of SQL syntax[15].

Rollback Protection Implementation:

In transactional systems: “A transaction consists of a sequence of query and/or update statements. The SQL standard specifies that a transaction begins implicitly when an SQL statement is executed. One of the following SQL statements must end the transaction: **Commit work**….**Rollback work**…”[15]. Because updates can corrupt transactional integrity, they need to be able to prevent partial failures[15]. If an invalid transaction update happens, the system uses rollback, which ensures that the database returns back to its previous valid state[15]. An example in this project is, if a user tries to change a “Refunded” transaction back to “Pending”, the stored procedure rejects the request and the transaction is rolled back completely, which helps to prevent inconsistent states, and ensures that financial records stay accurate and trustworthy.

Code Organization:

During development, the project was mainly done in 1-2 Python files, but was refactored into multiple Python files to improve maintainability and appear more professional. The final structure separates responsibilities across files. To support this, the source code also uses Docstrings and similar comments for code files.

*The Final Submission instructions mention that the Final Report is to be of 20+ pages(excluding, if applicable, source codes and screenshots). Due to this, the source code, including the Python files and SQL source code, will be included in the final submission ZIP file, rather than directly inside this main report document.

Experimental Results

Bcrypt Experimentation Difficulties:

- I decided to put the following sequence of experiments in the Experimental Results section of the report, although an underlying difficulty, due to the experimental research that was needed to solve the problem.

When doing hashing with bcrypt, there was an error: “AttributeError: module ‘bcrypt’ has no attribute ‘__about__’.” The underlying issue here was that brypt was causing problems, and the alternative to be bcrypt was to use argon2. Argon2 to would have caused me to change around configuration and alter in the middle of the project/ authentication phase, resulting in a slightly more complex setup, which was not really necessary for the project. I researched and uncovered that the attribute error mentioned above means that the bcrypt version I previously installed was too new for the version of passlib that I previously installed[16]. To expand, I learned that the root cause was

because the newer bcrypt versions have removed __about__,--version and passlib still continues to attempt gaining access to it[16]. This results in backend loading issues, and triggers other side errors such as the following: “ValueError: password cannot be longer than 72 bytes, truncate manually if necessary(e.g. my_password[:72])”. This is nothing I did incorrectly, rather, it was a compatibility issue between passlib and certain versions of bcrypt.

Due to the previous complications, I did some troubleshooting. I first opened the terminal in pie charm, from the .venv/project file, and deleted bcrypt and passlib completely by doing: “pip uninstall passlib bcrypt -y”.Following the uninstallations, I input the command “pip cache purge” to remove all items from pip’s local cache. I was hesitant to do the cache purge, especially since I have completed most of the project, but before I proceeded with the cache purge, I made sure to research what it does. From the official pip documentation, the cache stores package files, such as wheels and source distributions, that were previously downloaded in order to speed up installations in the future[17]. The pip command specifically removes wheel files, source distributions, and HTTP cache[17]. According to the documentation, it does not uninstall any packages, and it’s beneficial for reclaiming disk space, resolving installation errors, and ensuring fresh versions[17]. Once I knew it was safe to do, I executed the command.

Following the previous actions, I then reinstalled passlib by executing “pip install passlib”. Then, I installed an older version of bcrypt[17]. I then restarted Pycharm, ran the program/code, and it finished with no errors, enabling me to get my hashed password/ciphertext. I then copied the ciphertext, went to mySQL, and did the following:

DELETE FROM users WHERE username = ‘Dash’;

INSERT INTO users (username, email, password_hash)

VALUES (‘Dash’, ‘dashtestrun.com’, ‘ciphertext’);

I planned on doing the previous SQL code in order to temporarily reinsert the user/fix the test user in Python/temporarily in code, because I previously did testing on my endpoints without any authentication, just to make sure my backend logic functioned correctly before adding the extra security layers.

After I attempted to execute the previous MySQL code, I got an error, which in this case, was a good thing. The error code was the following: “cannot delete or update a parent row: a foreign key constraint fails(audit_log to users)”.

The error means that the code was trying to delete a user that is still being referenced in another table, which in this case is the table audit_log. This was actually a very good sign, because it informed me that my design is working/behaving correctly. It tells me that my database is enforcing referential integrity correctly. This error happened because in MySQL, I designed the following:

FOREIGN KEY (user_id)

REFERENCES users(id)

ON DELETE RESTRICT,

which means you cannot delete a user if they have audit logs, and/or if they have transactions. This design prevents losing audit history, breaking relationships, and data

inconsistency, which is considered professional level behavior. Due to this, instead of deleting the user, I just updated the password instead, using the following in MySQL:

UPDATE users

SET password_hash = (ciphertext)
'$2b$12$IvHK0S9HlLQ9h2myAs6aeuL6ZKm8E.GLnSbfxVszCvSVhzM5gNlwW'

WHERE username = 'Dash';

This code keeps all audit history, fixes the login, and attempts to match real-world systems.

In addition, my test user has a password called ‘testhash’ in the mySQL database, which was not a bcrypt hash, which would make the login fail. Hence, why I fixed the test user in python. After I made adjustments to the login logic, I did the following in Python so my data, at the time, would not break:

hashed = to_secure_password(“testhash”)

print(hashed)

This code would allow me to get the ciphertext needed from the plaintext(testhash), so I can copy and paste it from python into the MySQL code previously stated. These actions allowed my login end point to work/execute correctly, allowing me to get a token, and also ensuring that the password is not stored in plaintext. To elaborate, previously, if the database were to get leaked, it would have exposed passwords and it would have been easy to attack. After the adjustments, the passwords would now be protected and

attacks will now be computationally more expensive. This system now follows more industry standards that are used by banks, APIs, SaaS platforms, and authentication systems.

Local/FastAPI Security Testing:

Prior to adding the security measures to the system, it was very vulnerable to security threats. For example, previously, the user was able to create transactions from the create_transaction endpoint, along with making status updates to transactions through the update_transaction endpoint, both from the FastAPI application. However, this was purposely allowed to happen, temporarily for production, to test and clarify that those endpoints worked as they were intended, and to make sure that data entries were able to be inserted into the database correctly, before setting up authentication and any role-based access/authorization.

Now, after the implementation of the security measures, multiple real-world vulnerabilities have been mitigated. The system now protects against vulnerabilities such as authentication bypass, SQL injection, insecure credential storage, unauthorized access, and data integrity violations. The following is a detailed breakdown of what has/ is currently secure.

1. Authentication Bypass Mitigation

   Previously, the system depended on client supplied user identifiers, which allowed for possible impersonations by manually making changes to request data. This vulnerability was mitigated by implementing JWT-based

authentication, making sure that the user identities are verified through signed tokens instead of client input.

2. Token Forgery and Validation Protection

   JWT tokens have been securely generated using a server-side secret key and validated on every request. This helps to prevent attackers from forging or modifying tokens, because any tampering will result in token invalidation and request rejection. I proved this, locally, by attempting to change a character in the bearer token for authorization, and when I attempted to create and update transactions, an error occurred stating: code 401: Error: Unauthorized

   {“detail”: “Not Authenticated"}

3. Secure Password Storage

   Plain text password storage in the database was removed and replaced with bcrypt hashing and passlib, as mentioned previously. This change caused user credentials to never be stored in the database in readable form, reducing the risks that are linked with database compromise. Previously, in the database under the users table, in the password column, the test user’s password

   (‘ testhash’) was exposed fully. Now, it is hashed and stored as $2b$12$2…, successfully stored as ciphertext/hashed password text.

4. SQL Injection Protection

Malicious SQL injection attempts were prevented/reduced by the use of parameterized statements for all database queries interacting with user input. The parameterized statements make sure that user input is treated as data instead of code that can be executed, also previously mentioned.

5. Role-Based Access Control(RBAC)

A structured RBAC system was put in place, using roles and user_role mappings. Now, sensitive operations, like updating the status of transactions, are restricted to authorized roles only, which are either the admin or support user roles (IDs 1 and 2), preventing unauthorized privilege escalations.

6. Insecure Direct Object Reference(IDOR) Prevention

Previously, the system would allow users to specify identifiers, like user_id, in the API requests. Now, user identity is obtained directly from the validated JWT token, eliminating the possibility of accessing or modifying other user's data.

7. Business Logic Validation and State Enforcement

Strict validation rules were applied to status changes regarding transactions, making it to where only valid state transitions are allowed, as previously mentioned. This gets rid of logical inconsistencies and unauthorized transaction workflow manipulations.

8. Data Integrity Enforcements Via Constraints

Within the database schema, foreign key constraints, along with relational integrity rules, were enforced. These constraints eliminate invalid operations, like deleting users with associated audit records or transactions, supporting overall system consistency. This is also supported by the previous example I mentioned, where I first attempted to delete the test user and reinsert the hashed password, resulting in an error, causing me to do an update and insert statement instead.

9. Unauthorized Endpoint Access Protection

   All sensitive API endpoints are protected and now require valid authentication tokens, as mentioned previously. Now, requests without proper authorization are rejected which makes sure that only authenticated users can access system functionality.

10. Audit Logging and Traceability

    An audit logging process was put in place to track/monitor important system actions, including the creation of transactions and updates to transactions. This supports accountability and traceability, allowing system behaviors to be analyzed and monitored, also previously mentioned.

Security Testing

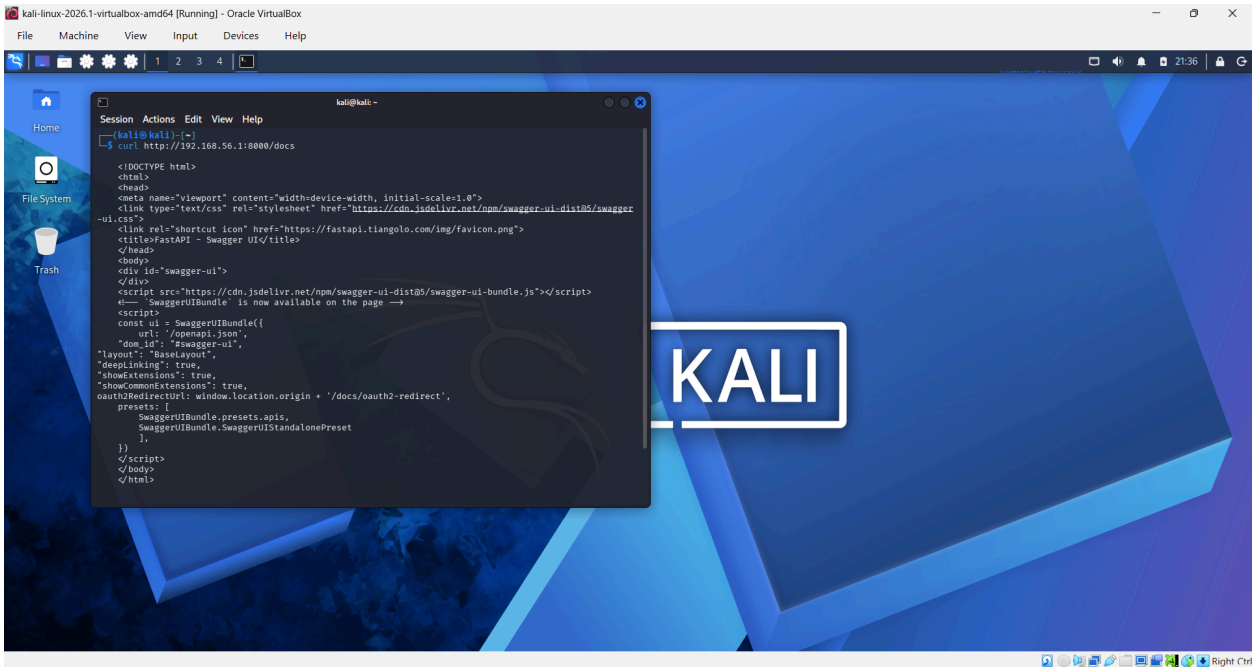
For this phase of the project, I experimented and conducted security testing utilizing tools from Kali Linux. Kali Linux is a unique operating system that consists of various penetration testing, security auditing, and forensics tools built into it[18]. The tools that come pre-installed with Kali Linux are designed to aid professionals in analyzing and

securing computer networks and systems[18]. In addition, I have become somewhat familiar, and have previously installed Kali Linux for previous courses, which include CSC-410-Database Management Systems, and CSC-360-Computer System Security. In order to conduct security/penetration testing with Kali Linux, I previously downloaded a pre-built Kali Linux “image” for Oracle virtualbox. Also, to ensure that all tools were currently up to date, I ran the command “sudo apt update && sudo apt upgrade -y”.

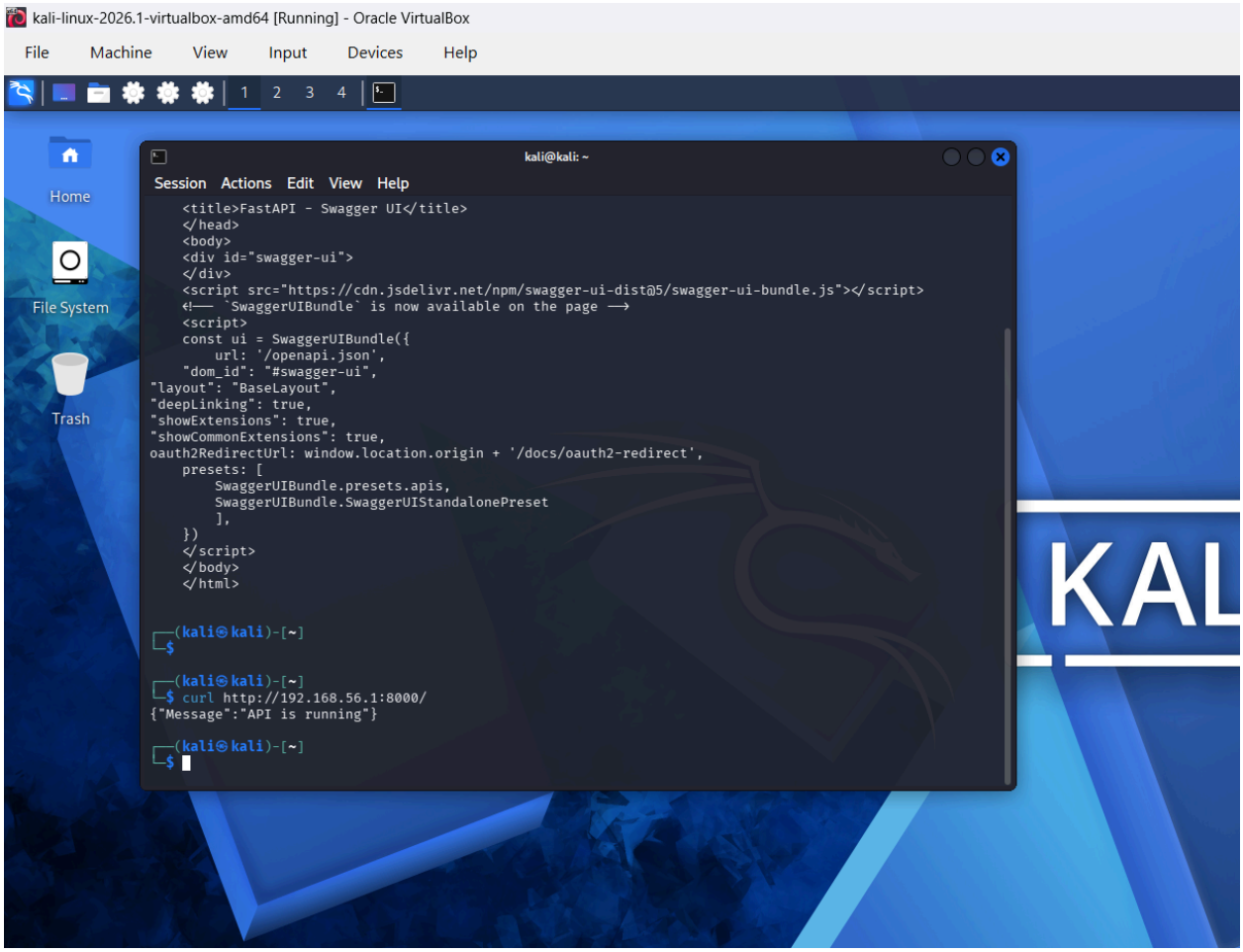
In order to begin testing the security, I also needed to make sure that FastAPI was reachable from the Kali Linux virtual machine. Previously, the API ran with the following command: “uvicorn main:app - - reload”. This command, however, only allows FastAPI to run locally. In addition, by default, FastAPI and MySQL were bound to 127.0.0.1(localhost), which cannot be accessed from a virtual machine. Due to that, I needed to set the host flag to 0.0.0.0[19]. :”The I.P. address 0.0.0.0 means all IPv4 addresses on the local machine”[19]. This means that if a host has two IP addresses, which in my case it does, and if the server running on the host is listening on 0.0.0.0, then it will be able to be accessed/reached at both of those IPs[19]. Therefore, the new command to run the API would be the following: “uvicorn main:app - - host 0.0.0.0 - -port 8000”[19].

The next step after making FastAPI reachable from Kali was to find my host IP. I'm using windows, so I opened the Powershell and entered “ipconfig”. Once I made sure that FastAPI was reachable and obtained the IP of the host machine, the next step was to confirm Kali Linux could actually reach it. To do that, I ran the following command in the Kali Linux terminal: “curl http://”my_ip”:8000/docs” It enabled me to view HTML

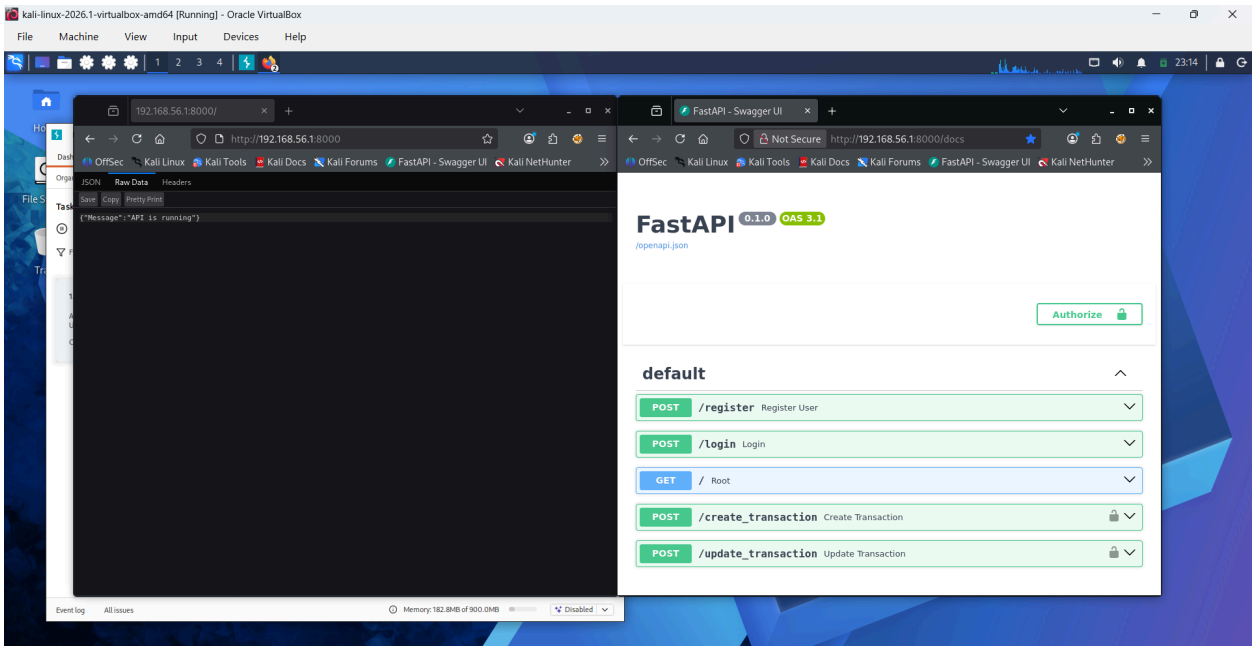
output from the Swagger docs, which meant the connection was successful. The following image shows the successful connection.



The next step that followed was to test the root endpoint, so in the Kali Linux terminal I ran the following: “curl http://”my_ip”:8000/”. The output returned the correct message, which means that communication with the API was successful, shown in the following images.


kali-linux-2026.1-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Home
File System
Trash
kali@kali: ~
Session Actions Edit View Help
<title>FastAPI - Swagger UI</title>
</head>
<body>

<script src="https://cdn.jsdelivr.net/npm/swagger-ui-dist@5/swagger-ui-bundle.js"></script>
<!-- `SwaggerUIBundle` is now available on the page -->
<script>
const ui = SwaggerUIBundle({
url: '/openapi.json',
"dom_id": "#swagger-ui",
"layout": "BaseLayout",
"deepLinking": true,
"showExtensions": true,
"showCommonExtensions": true,
oauth2RedirectUrl: window.location.origin + '/docs/oauth2-redirect',
presets: [
SwaggerUIBundle.presets.apis,
SwaggerUIBundle.SwaggerUIStandalonePreset
],
})
</script>
</body>
</html>
(kali㉿kali)-[~]
$
(kali㉿kali)-[~]
$ curl http://192.168.56.1:8000/
{"Message":"API is running"}
(kali㉿kali)-[~]
$


kali-linux-2026.1-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
192.168.56.1:8000/
http://192.168.56.1:8000
OffSec Kali Linux Kali Tools Kali Docs Kali Forums FastAPI - Swagger UI Kali NetHunter
JSON Raw Data Headers
Save Copy Pretty Print
{"Message":"API is running"}
FastAPI - Swagger UI
Not Secure http://192.168.56.1:8000/docs
FastAPI 0.1.0 OAS 3.1
/openapi.json
Authorize
default
POST /register Register User
POST /login Login
GET / Root
POST /create_transaction Create Transaction
POST /update_transaction Update Transaction

Once Kali and FastAPI were communicating/connected successfully, I was clear to begin simulating test attacks.

(Manual Attacks):

Attack Test #1: Unauthorized Access

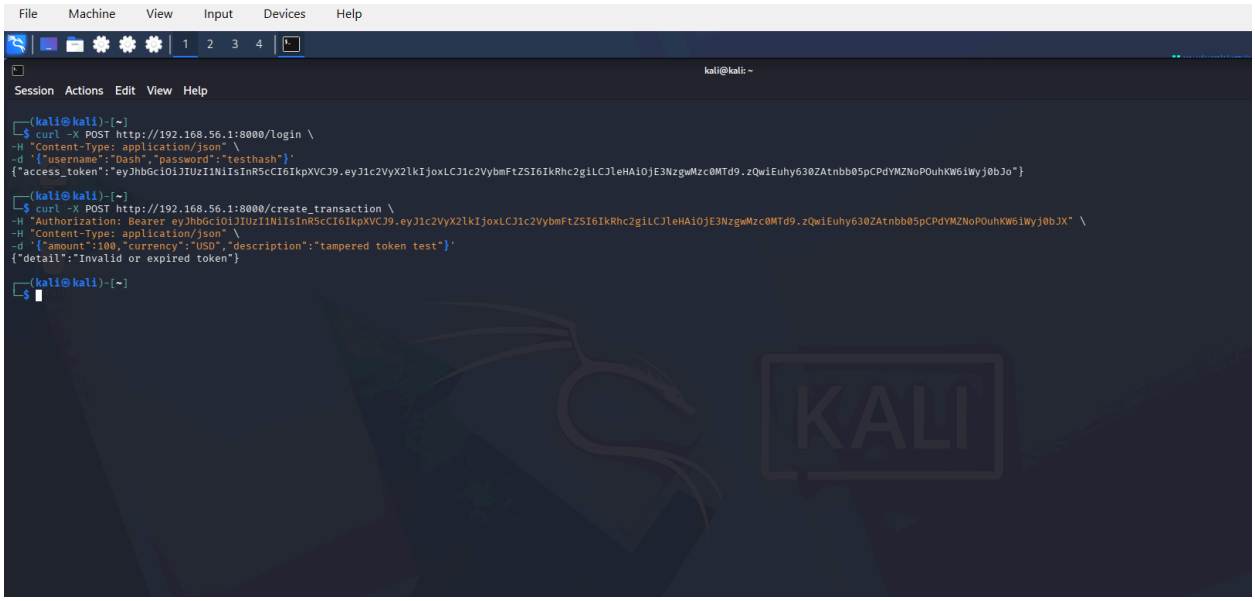
- The main goal of this attack was to prove that a user cannot access protected endpoints without proper authentication, which validates the JWT protection
- This attack attempts to create a transaction without logging in, and without sending a token. (The image shows the process.)



- This attack simulation proves that the endpoint is protected, JWT is required, and no anonymous access is granted/allowed. As a result of the attack, the system correctly rejected the request with an authentication error, confirming that protected routes enforce token-based access control.

Attack #2: JWT Token Tampering

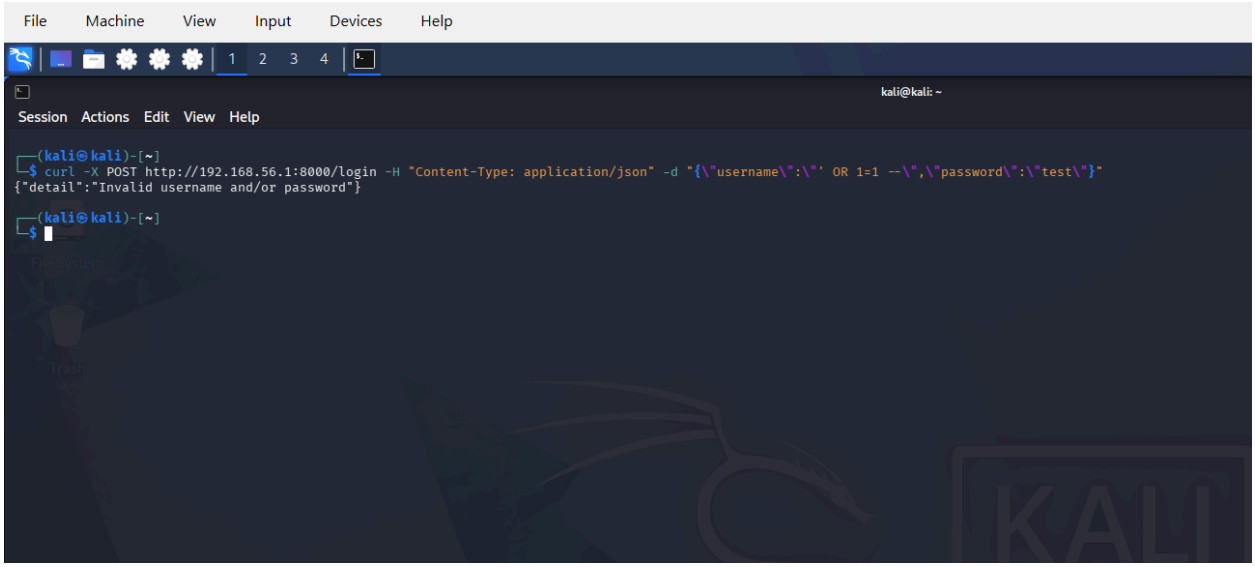
- The main goal of this attack is to prove/demonstrate that modified or fake tokens are rejected by the system. If successful, this attempt will validate token signature verification, the Integrity of tokens, and authentication security. The first step of this attack is to obtain a valid token using the normal login endpoint. The next step is to take the access token and break it on purpose by changing just one character. After that, the next step is to use the broken/fake token to try and create a transaction. (Full screenshot of simulated attack below.)



  - The following output that was captured means that the attack failed/was blocked successfully, concluding that JWT signature validation works, and forged tokens are rejected.

Attack Test #3: SQL Injection(Manual Test)

- The goal of this attack was to prove that malicious SQL input cannot bypass login authentication. doing this will validate parameterized queries, secure database interactions, along with injection resistance. The following bypass method, ‘OR 1=1 - -, is a classic payload that attempts to force login success[20]. Additionally, the curl command used was influenced by the following source: apisec.ai[21]. (Image below demonstrates the process.)



  - To summarize, SQL injection happens when a user input is treated as part of the SQL command, instead of being handled as data, as mentioned prior. This is a vulnerability that can be made present when using string concatenation to build SQL queries dynamically[22]. An example, which is unsafe, is the following:

    “query = “SELECT * FROM users WHERE username = ‘ “ + username + “ ‘ ”

    cursor.execute(query)

For that example, if an attacker was to enter malicious input as the username such as: ‘ OR 1=1 - -, then the final query turns into:

SELECT * FROM users WHERE username = ‘ ‘ OR 1=1 - -’[22]

According to various sources, this is dangerous and not advised because the condition OR 1=1 is always going to be true, and as a result, the query could return all of the users in the database, allowing an attacker to bypass authentication without using a legitimate password[20,21,22].

Prior to building the mySQL database and connecting it with python code, I researched known/common security vulnerabilities and became aware of this problem. Therefore, additional research led me to utilize parameterized queries, also called prepared statements, which was mentioned multiple times previously, which is also demonstrated in the python code.

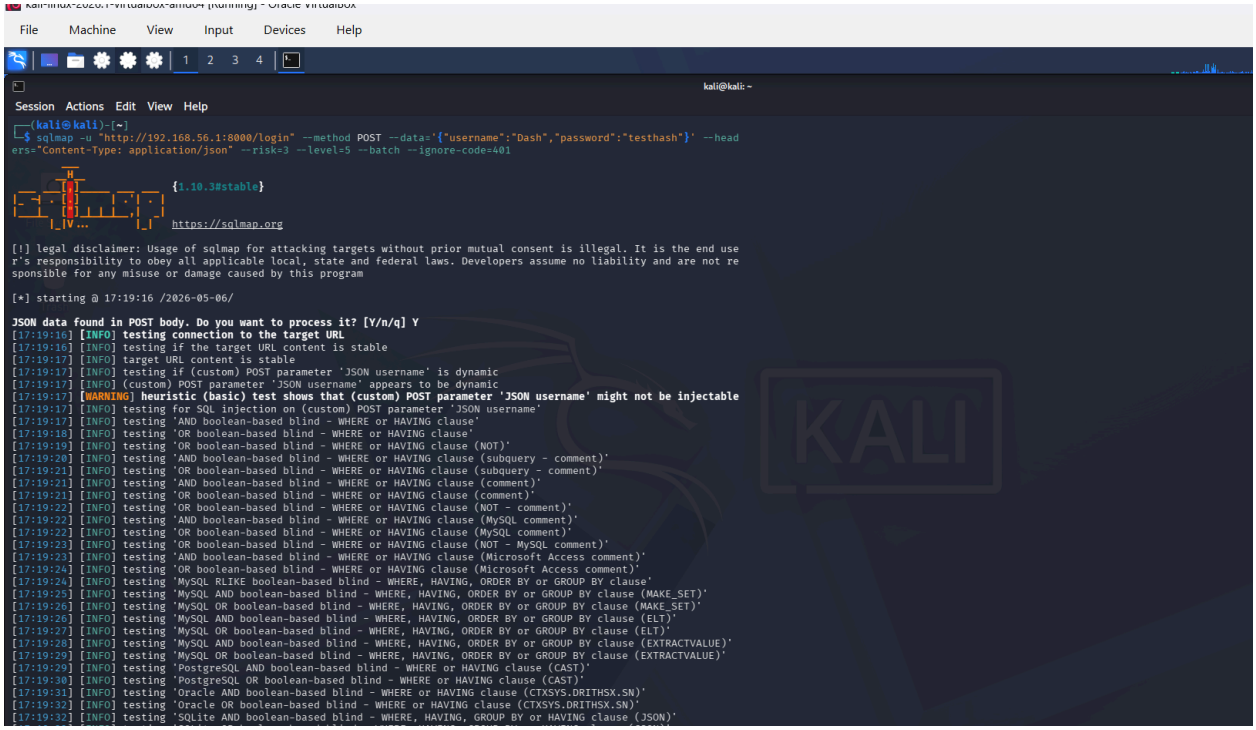
In this approach, as mentioned previously, I use the “%s” to act as a placeholder for the user's input. As a result, MySQL safely binds the value later as pure data and not as executable SQL code, rather than manually inserting the input into the SQL statement[15].

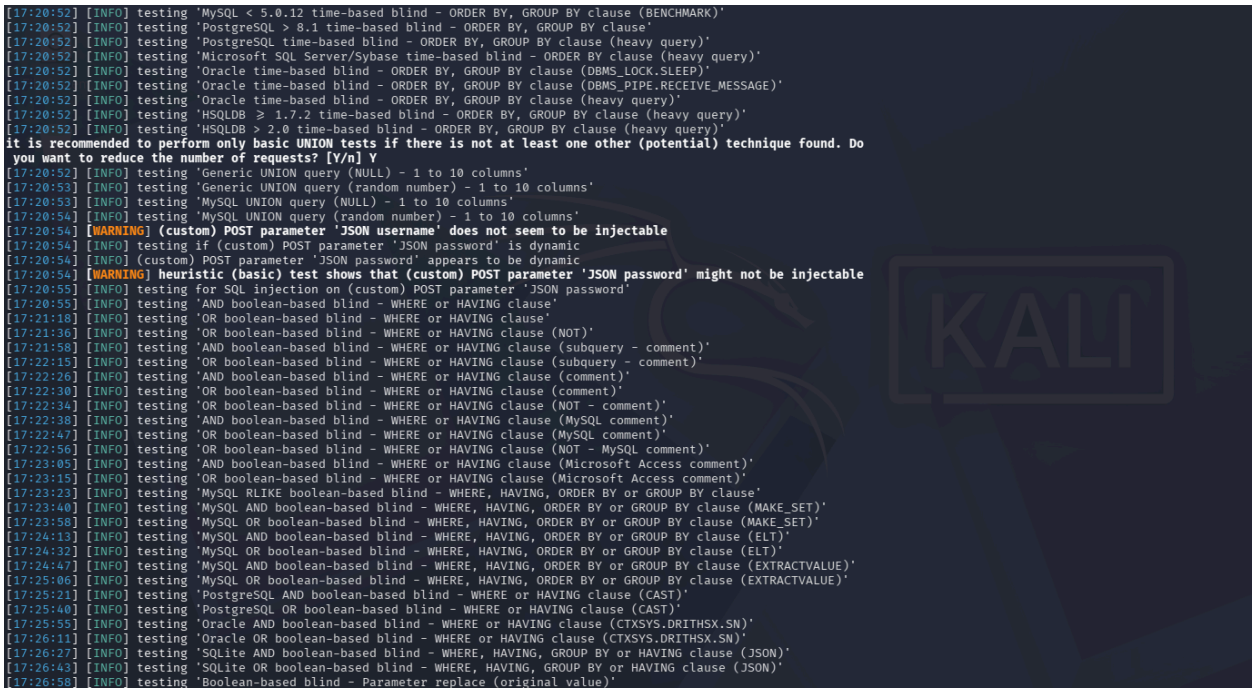
Therefore, if an attacker were to enter ‘ OR 1=1 - -;, MySQL will see it as literal text instead of SQL logic. Also, instead of changing the structure of the query, the database basically looks for a username that is named ‘ OR 1=1 - -, which does not exist, so the system correctly returns: “Invalid username and/or password”, which means the attack failed successfully.

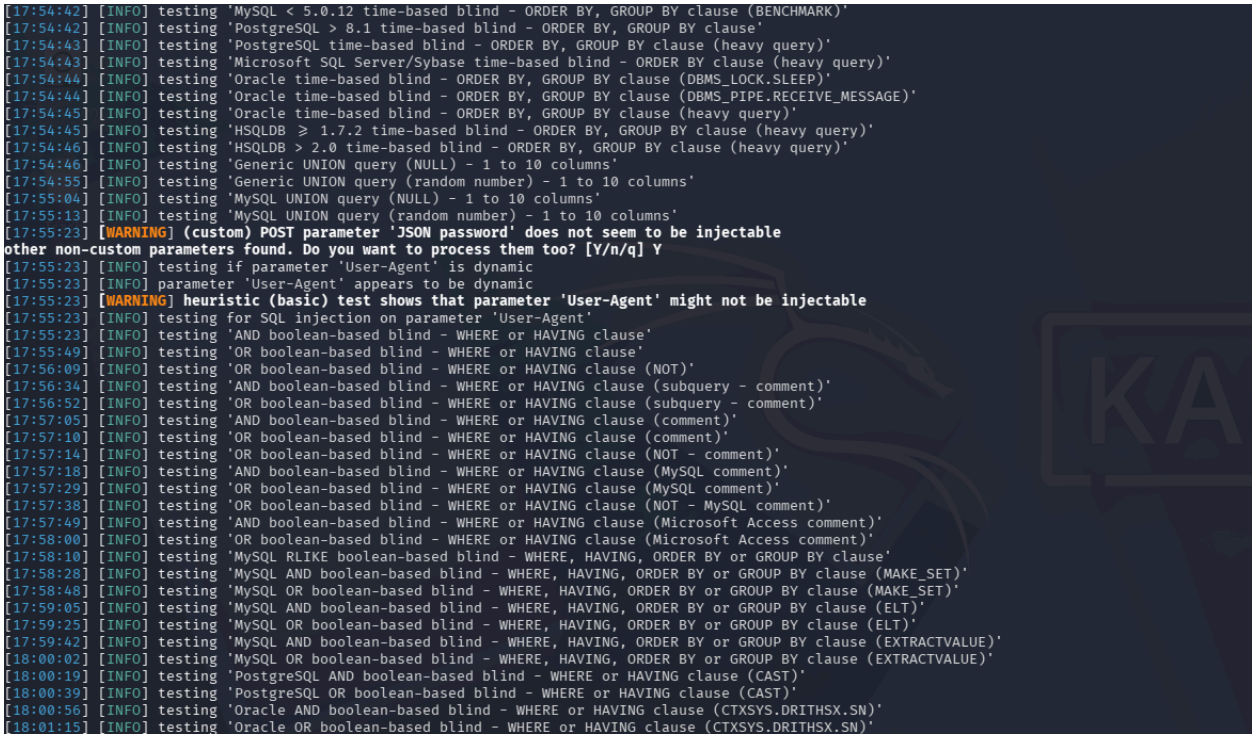
(Automated Attacks):

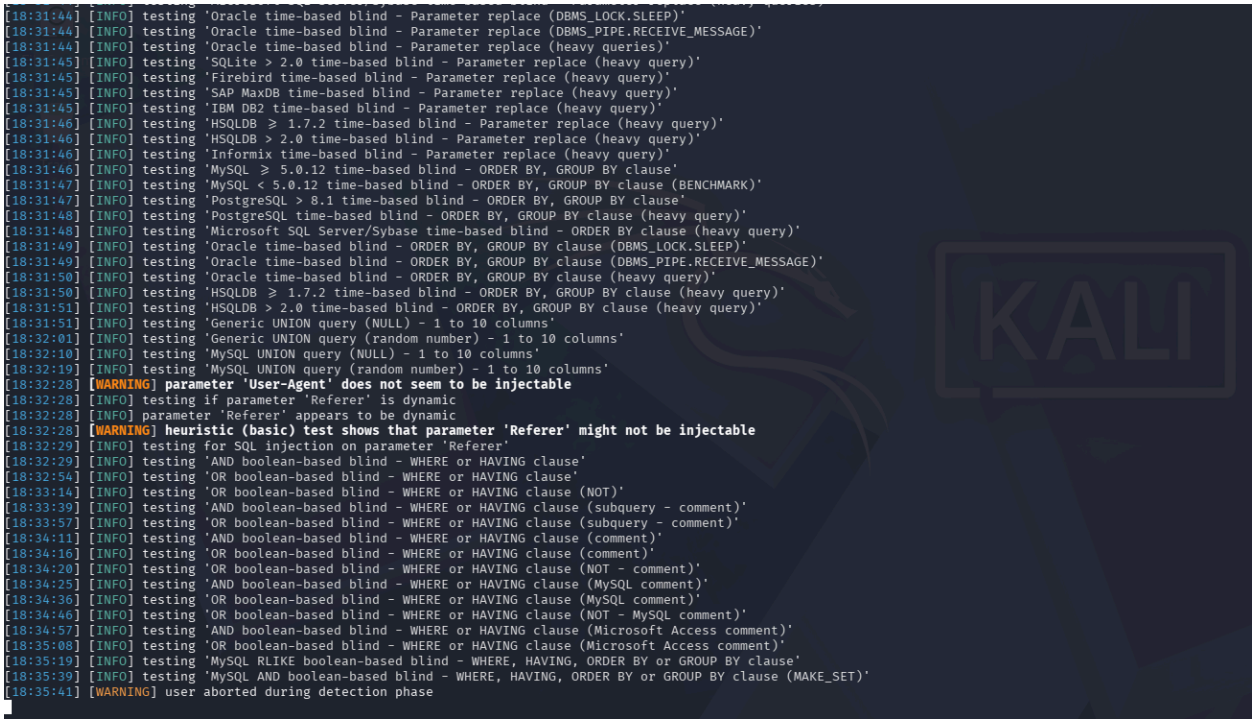
Attack Test #4: Automated SQL Injection With SQLMap

- Now that manual SQL injection has been tested and proven to not be successful against the system, the goal of this next attack is to prove that automated attack tools cannot expose SQL injection vulnerabilities. This will also validate parameterized queries, secure backend design, and resistance to/against real-world attack tools. The following automated attack will be used with the SQLMap tool included in/with the Kali Linux VM. “SQLMap is a powerful, free, open-source penetration testing tool that comes pre-installed in Kali Linux. It is a tool, used by security professionals conducting web application assessments, that automates the process of detecting and exploiting SQL injection vulnerabilities in database systems”[23].I will be targeting my login endpoint for this attack simulation. The following attack simulation code/commands are influenced/adapted by the source darkmarc.substack.com[23]. (Images below show the process and results).


File Machine View Input Devices Help
1 2 3 4
kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sqlmap -u "http://192.168.56.1:8000/login" --method POST --data='{"username":"Dash","password":"testhash"}' --headers="Content-Type: application/json" --risk=3 --level=5 --batch --ignore-code=401
{1.10.3#stable}
https://sqlmap.org
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[*] starting @ 17:19:16 /2026-05-06/
JSON data found in POST body. Do you want to process it? [Y/n/q] Y
[17:19:16] [INFO] testing connection to the target URL
[17:19:16] [INFO] testing if the target URL content is stable
[17:19:17] [INFO] target URL content is stable
[17:19:17] [INFO] testing if (custom) POST parameter 'JSON username' is dynamic
[17:19:17] [INFO] (custom) POST parameter 'JSON username' appears to be dynamic
[17:19:17] [WARNING] heuristic (basic) test shows that (custom) POST parameter 'JSON username' might not be injectable
[17:19:17] [INFO] testing for SQL injection on (custom) POST parameter 'JSON username'
[17:19:17] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[17:19:18] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause'
[17:19:19] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT)'
[17:19:20] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[17:19:21] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[17:19:21] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (comment)'
[17:19:21] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (comment)'
[17:19:22] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - comment)'
[17:19:22] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[17:19:22] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[17:19:23] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[17:19:23] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (Microsoft Access comment)'
[17:19:24] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (Microsoft Access comment)'
[17:19:24] [INFO] testing 'MySQL RLIKE boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause'
[17:19:25] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MAKE_SET)'
[17:19:26] [INFO] testing 'MySQL OR boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MAKE_SET)'
[17:19:26] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (ELT)'
[17:19:27] [INFO] testing 'MySQL OR boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (ELT)'
[17:19:28] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[17:19:29] [INFO] testing 'MySQL OR boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[17:19:29] [INFO] testing 'PostgreSQL AND boolean-based blind - WHERE or HAVING clause (CAST)'
[17:19:30] [INFO] testing 'PostgreSQL OR boolean-based blind - WHERE or HAVING clause (CAST)'
[17:19:31] [INFO] testing 'Oracle AND boolean-based blind - WHERE or HAVING clause (CTXSYS.DRITHSX.SN)'
[17:19:32] [INFO] testing 'Oracle OR boolean-based blind - WHERE or HAVING clause (CTXSYS.DRITHSX.SN)'
[17:19:32] [INFO] testing 'SQLite AND boolean-based blind - WHERE, HAVING, GROUP BY or HAVING clause (JSON)'


[17:20:52] [INFO] testing 'MySQL < 5.0.12 time-based blind - ORDER BY, GROUP BY clause (BENCHMARK)'
[17:20:52] [INFO] testing 'PostgreSQL > 8.1 time-based blind - ORDER BY, GROUP BY clause'
[17:20:52] [INFO] testing 'PostgreSQL time-based blind - ORDER BY, GROUP BY clause (heavy query)'
[17:20:52] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind - ORDER BY clause (heavy query)'
[17:20:52] [INFO] testing 'Oracle time-based blind - ORDER BY, GROUP BY clause (DBMS_LOCK.SLEEP)'
[17:20:52] [INFO] testing 'Oracle time-based blind - ORDER BY, GROUP BY clause (DBMS_PIPE.RECEIVE_MESSAGE)'
[17:20:52] [INFO] testing 'Oracle time-based blind - ORDER BY, GROUP BY clause (heavy query)'
[17:20:52] [INFO] testing 'HSQLDB ≥ 1.7.2 time-based blind - ORDER BY, GROUP BY clause (heavy query)'
[17:20:52] [INFO] testing 'HSQLDB > 2.0 time-based blind - ORDER BY, GROUP BY clause (heavy query)'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] Y
[17:20:52] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[17:20:53] [INFO] testing 'Generic UNION query (random number) - 1 to 10 columns'
[17:20:53] [INFO] testing 'MySQL UNION query (NULL) - 1 to 10 columns'
[17:20:54] [INFO] testing 'MySQL UNION query (random number) - 1 to 10 columns'
[17:20:54] [WARNING] (custom) POST parameter 'JSON username' does not seem to be injectable
[17:20:54] [INFO] testing if (custom) POST parameter 'JSON password' is dynamic
[17:20:54] [INFO] (custom) POST parameter 'JSON password' appears to be dynamic
[17:20:54] [WARNING] heuristic (basic) test shows that (custom) POST parameter 'JSON password' might not be injectable
[17:20:55] [INFO] testing for SQL injection on (custom) POST parameter 'JSON password'
[17:20:55] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[17:21:18] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause'
[17:21:36] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT)'
[17:21:58] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[17:22:15] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[17:22:26] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (comment)'
[17:22:30] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (comment)'
[17:22:34] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - comment)'
[17:22:38] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[17:22:47] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[17:22:56] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[17:23:05] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (Microsoft Access comment)'
[17:23:15] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (Microsoft Access comment)'
[17:23:23] [INFO] testing 'MySQL RLIKE boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause'
[17:23:40] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MAKE_SET)'
[17:23:58] [INFO] testing 'MySQL OR boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (MAKE_SET)'
[17:24:13] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (ELT)'
[17:24:32] [INFO] testing 'MySQL OR boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (ELT)'
[17:24:47] [INFO] testing 'MySQL AND boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[17:25:06] [INFO] testing 'MySQL OR boolean-based blind - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[17:25:21] [INFO] testing 'PostgreSQL AND boolean-based blind - WHERE or HAVING clause (CAST)'
[17:25:40] [INFO] testing 'PostgreSQL OR boolean-based blind - WHERE or HAVING clause (CAST)'
[17:25:55] [INFO] testing 'Oracle AND boolean-based blind - WHERE or HAVING clause (CTXSYS.DRITHSX.SN)'
[17:26:11] [INFO] testing 'Oracle OR boolean-based blind - WHERE or HAVING clause (CTXSYS.DRITHSX.SN)'
[17:26:27] [INFO] testing 'SQLite AND boolean-based blind - WHERE, HAVING, GROUP BY or HAVING clause (JSON)'
[17:26:43] [INFO] testing 'SQLite OR boolean-based blind - WHERE, HAVING, GROUP BY or HAVING clause (JSON)'
[17:26:58] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'





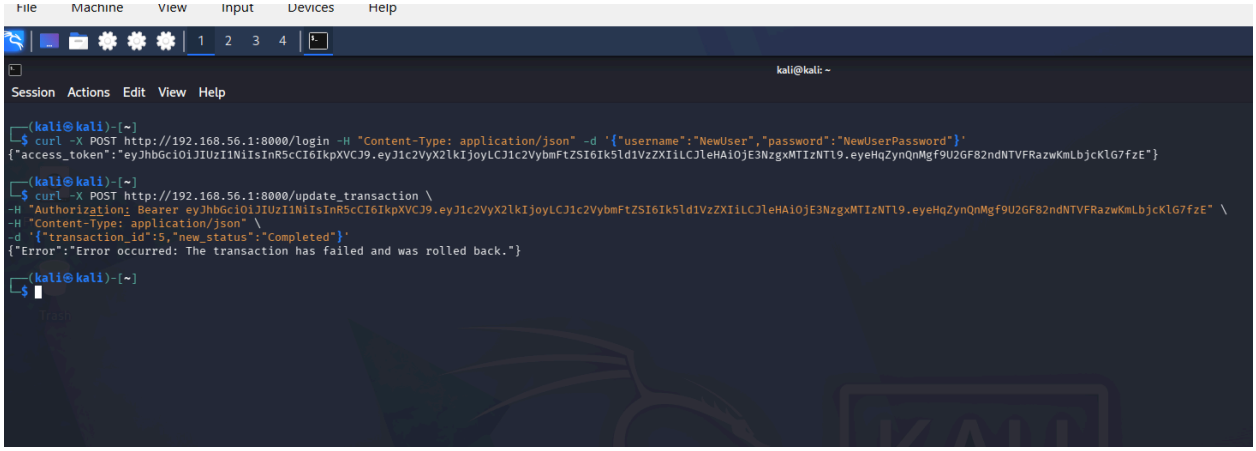
- To give context, this attack was considered a deep test, risk=3, level=5, which took over an hour and a half. To summarize the attack/screenshots,

automated SQL injection testing was carried out using SQLMap against the login endpoint. The main/primary authentication parameters including username and password were tested and determined to be non-injectable. In addition, HTTP headers, such as User-Agent, were also tested and showed no injection vulnerabilities. Due to the main/critical input fields being confirmed as secure, continued/extra header testing was stopped after sufficient validation.

Attack Test #5: RBAC Authorization Bypass

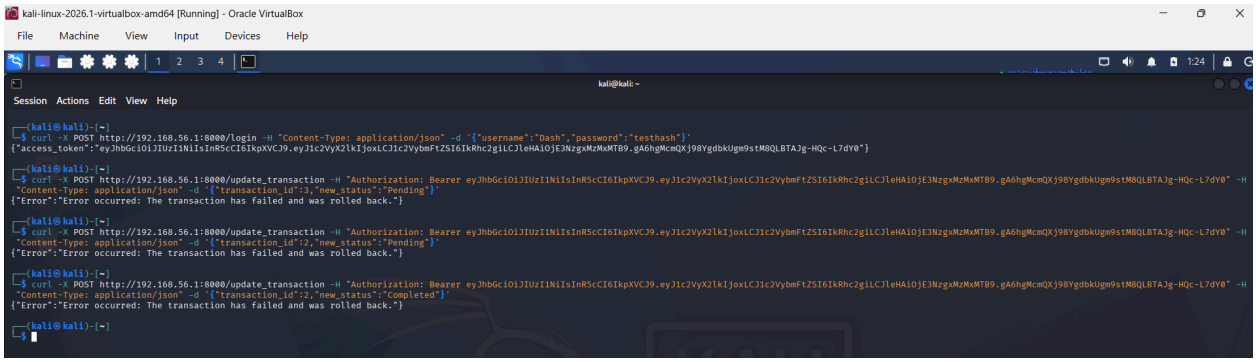
- The main goal of this attack was to try and prove that a normal customer cannot perform admin/support only actions. This attack, if successful, validates role-based access control(RBAC) principle of least privilege, and stored procedure authorization checks. Also, this test, along with some of the previous ones, were carried out locally. The tests seem redundant, however, the local tests were mainly to test the logic/verify the logic. Performing these tests from the Kali Linux terminal enables me to test perimeter verifications. It is also to make sure that the protection is not just a browser-level “fluke”/”front”, and that it actually holds up when an external, non-browser actor attempts to bypass and go directly after the endpoint.
- For this attack, I will log in as a valid/existing customer in the database, obtain the JWT token, and attempt to update a transaction status all from the Kali Linux terminal.(The following image shows the entire process, which resulted in an error, which is the generic/custom message set). This means that the RBAC

protection worked and the security results were successful, and the customers are not able to update the transaction.



Attack Test # 6: Invalid Transaction State Transition

- The main goal of this test is to prove and make sure that transactions cannot move into invalid states. For example, a transaction should not be able to go from “Completed” back to “Pending”, and from a “Refunded” state back to “Completed” or a “Pending” state. This will also validate the business logic enforcement, the Integrity of transactions, along with workflow security. In the database, transaction_id 2 is in a “Refunded” status and transaction_id 3 is set to a “Completed” status.
- For this test, I will use an admin token and attempt invalid transitions.(The following image shows the full process.)



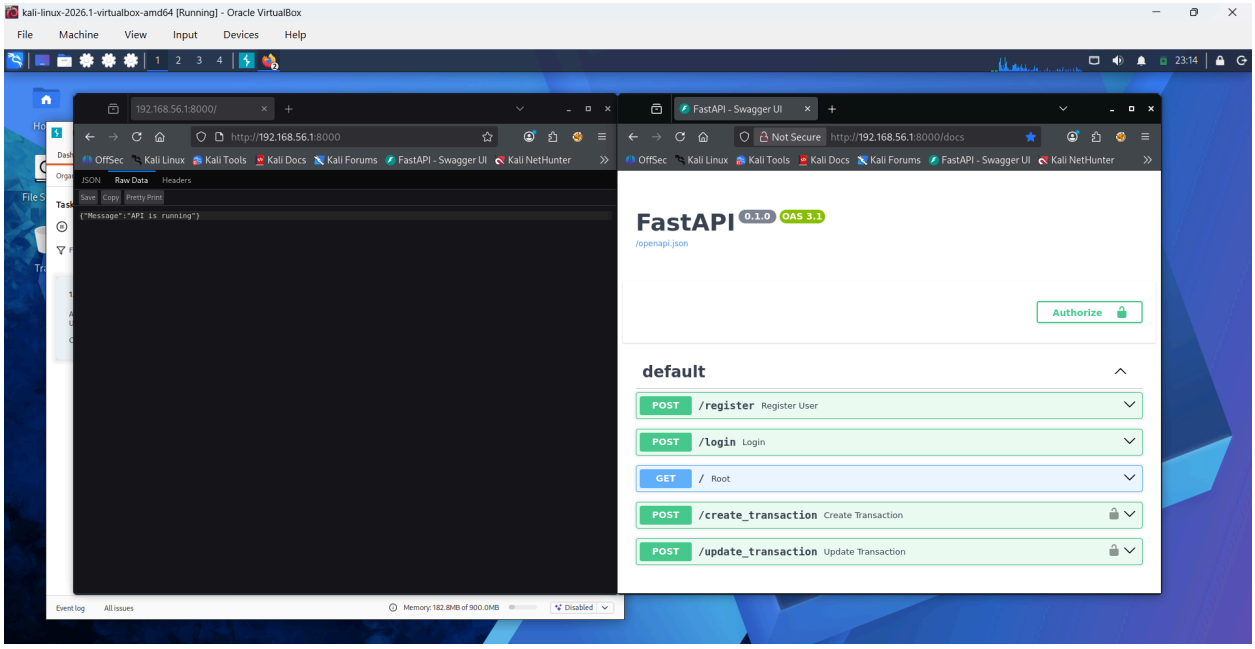
- As shown in the image, all invalid transaction state transitions resulted in an error output. This result means that invalid transitions are/were blocked and the transaction rules were successfully enforced, highlighting that the system also protects business integrity. To conclude, the system successfully enforced workflow rules, rollback protection, financial business logic validation, and transaction integrity due to the system's stored procedure logic that enforces a strict transaction lifecycle by design.

Attack Test #7: BurpSuite Security Testing

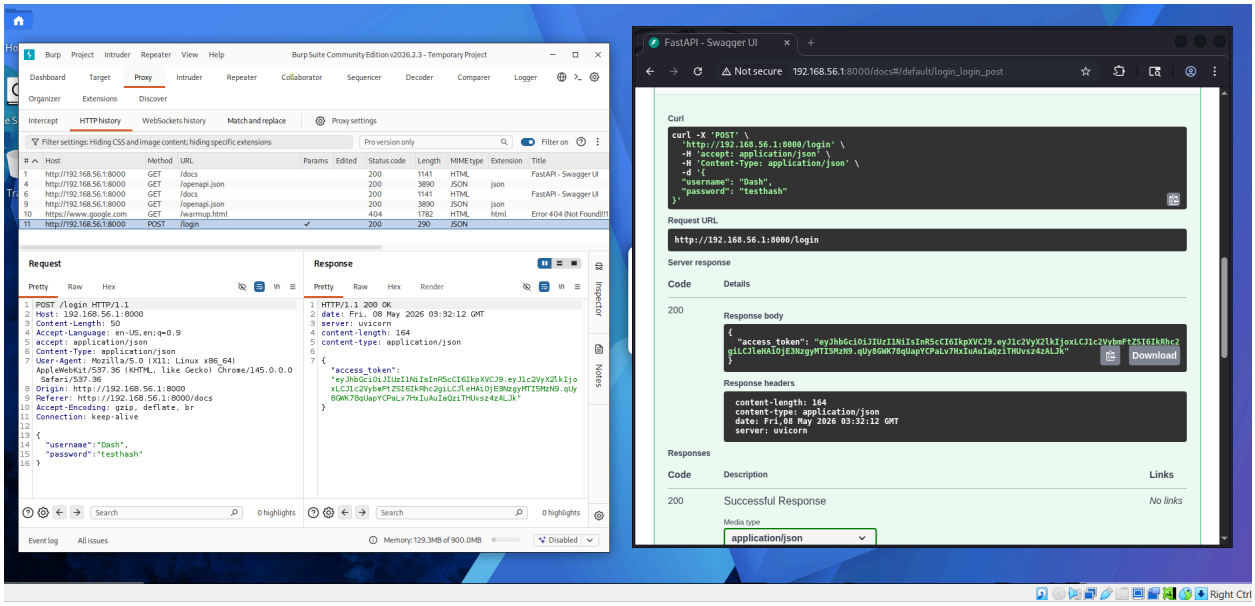
For these next series of attacks, I will be performing security/penetration testing on the project/ FastAPI system using BurpSuite in the Kali Linux virtual machine. From a previous course, CSC-360- Computer System Security, I became familiar with some basic tools in BurpSuite, performing an in class/live demo with it. BurpSuite is a platform used for conducting security tests on web applications, acting as a “man-in-the-middle” proxy to intercept, read/view, and modify/manipulate http/s traffic between a browser and a server[24]. The main goal behind the following tests are to have BurpSuite

intercept FastAPI requests, inspect JWTs and attempt to modify requests, and to test authorization/security controls.

- To make sure the connection worked, I opened the browser in Kali to get access to Swagger and FastAPI. (The following image confirms.)



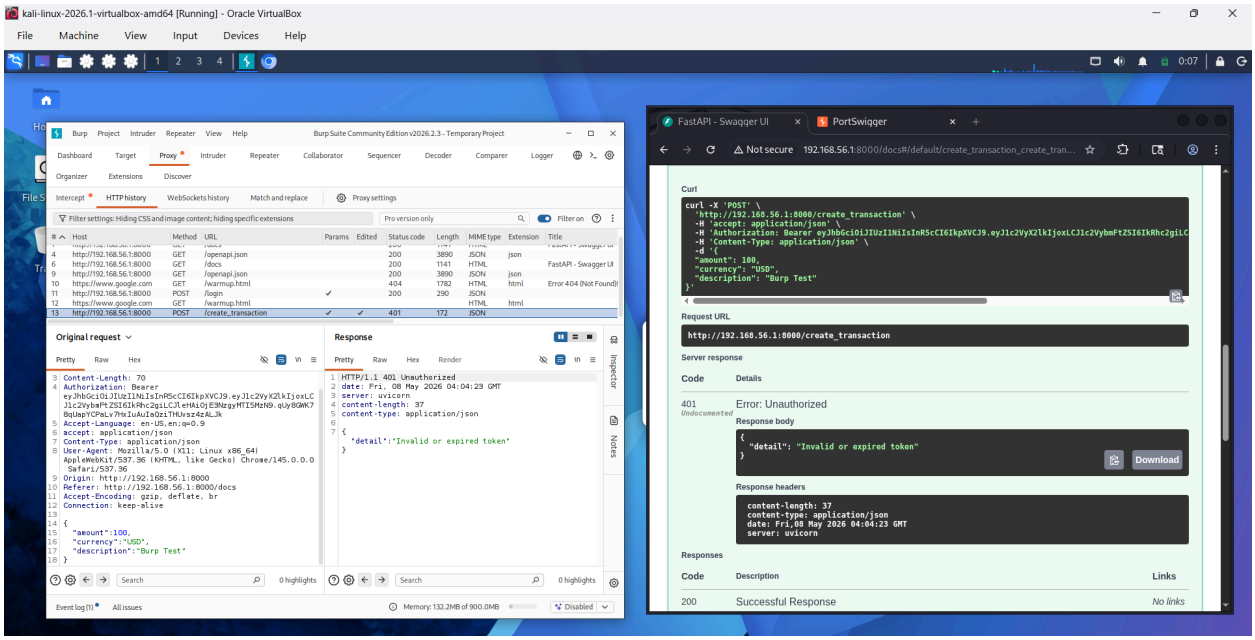
- I then launched BurpSuite, and began the first test(first Burp test), which is to intercept a login request. For this test, I am using the proxy intercept. The main objective is to verify that authentication requests could be inspected in transit, and to confirm that the API correctly handles structured JSON login requests. (Image of intercepted request below.)



- The image shows that Burp successfully intercepted the request(Login Credentials) and the server response(access token). This test mainly confirmed that login endpoints work, authentication succeeds, JWT generation works, and requests can be analyzed during testing.

Attack Test #8: JWT Tampering Attack

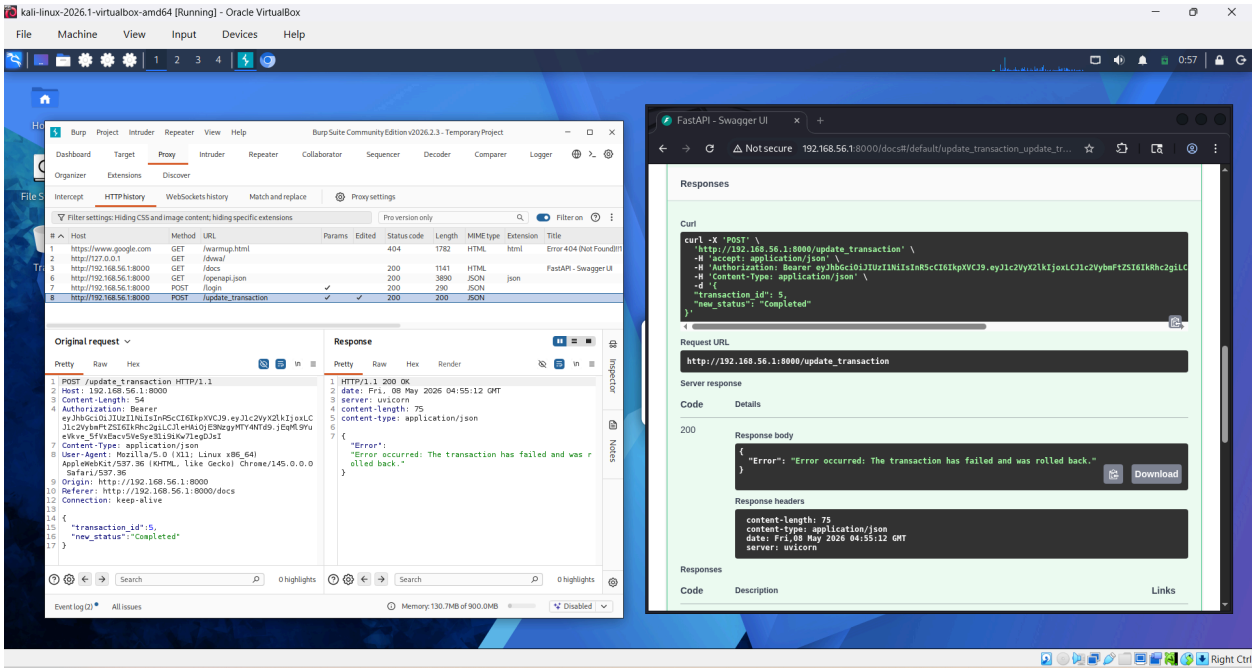
- The goal of this test was to prove, in Burp, that modified JWT tokens are rejected. This test validates token signature verification, anti-forgery protection, and authorization security. For this test, I am using the protected endpoint, create_transaction, while being authorized with the valid JWT. I will attempt to create a transaction, intercept the request in Burp, modify the JWT slightly to simulate token tampering, then forward the request. (The image of the response is below.)



- The image above shows the original request that was intercepted, then after forwarding the tampered token, the response was: “Invalid or expired token”.
- This test confirms that JWT signature validation works, and forged tokens are successfully rejected. This simulates a real attacker attempting to forge/alter a JWT, and the system successfully defending the attack.

Attack Test #9: Invalid Transaction Status Transition

- For this Burp test, I will login as admin and authorize normally. I will then attempt to update a transaction from “Pending” to “Completed” in FastAPI, which is a valid action, and intercept and change “Completed” to Refunded”, which should be invalid based on the stored procedure. (Image of the process below.)



- After forwarding the modified/edited request, the stored procedure rejected the request and triggered a rollback, returning the error message above, indicating a failed/rolled back transaction.
- This confirms that business logic validation was enforced at the database layer, rather than just depending on frontend restrictions. Even though requests were intercepted and modified in transit, unauthorized transaction status manipulation was successfully prevented/stopped.

Summary of Experimental Results:

- Based off of the security tests that were conducted on the system, both manual and automated, the project now includes backend security, database security, and penetration testing.

- Regarding backend security, the system includes/supports JWT authentication, password hashing, secure token validation, and protected API endpoints.
- On the topic of database security, the system includes parameterized queries, SQL injection prevention, RBAC enforcement, successful transaction rollback protection, and stored procedure security.
- On the element of penetration testing, the system underwent and successfully defended against unauthorized access testing, JWT tampering tests, manual SQL injection testing, automated SQLMap testing, automated BurpSuite testing, RBAC bypass testing, and business logic testing.

Discussion and Conclusion

Was the Approach Appropriate?

- Yes, I believe that the chosen approach was able to successfully address the identified security problems. By choosing/utilizing FastAPI, JWT, bcrypt, parameterize queries and MySQL stored procedures, a strong security foundation was able to be created, while also remaining realistic for a capstone project. In addition, security practices were enforced at multiple layers, which include the front end, the API, and the database. To conclude, this approach is also appropriate for transactional systems, specifically in this case financial systems.

Difficulties Faced:

The main challenges that I faced during this project, also previously mentioned and explained prior, included:

- **Passlib and bcrypt compatibility issues:**
    - During implementation, there were compatibility issues between bcrypt and Passlib because of version differences, which caused password hashing and verification errors, until the right package versions were installed and configured correctly.
- **JWT testing:**
    - Testing JWT authentication was more complex than standard logins, because each protected endpoint required valid token generation, proper bearer token formatting, expiration handling, and role verification before requests could be processed.
- **MySQL foreign key deletion restrictions:**
    - MySQL foreign key constraints created difficulties during testing because related records could not be deleted freely, which was also a good thing, requiring careful ordering of inserts, updates, and deletions in order to avoid violating integrity and maintaining consistent relational data.

Possible Methodology Improvements and/or Further Work Includes:

- Multi-factor authentication
- Admin dashboard
- Fraud detection system

- Email verification
- Cloud deployment using AWS
- Advanced analytics dashboard

What was Learned:

Throughout the project, it allowed me to gain practical and hands-on experience in API development, security knowledge and practices including penetration testing, SQL injection, authentication and authorization techniques, database design, auditing practices, networking, and full-stack integration. It also enabled me to go through an extensive research phase where I learned more about all of the topics mentioned above, in addition to other topics I have learned over the course of my education and outside interests/projects, and how they can all be utilized to work together to achieve a specific goal.

Citations/References

- OWASP Foundation. *OWASP Top 10 Web Application Security Risks.* *https://owasp.org/www-project-top-ten/* [1]
- https://www.lastingdynamics.com/blog/python-for-backend/ [2].
- https://fastapi.tiangolo.com/ [3]
- https://fastapi.tiangolo.com/reference/ [4]
- https://fastapi.tiangolo.com/reference/security/ [5]
- https://pydantic.dev/docs/validation/latest/get-started/ [6]
- https://dev.mysql.com/doc/connector-python/en/ [7]
- https://python-jose.readthedocs.io/en/latest/ [8]
- https://passlib.readthedocs.io/en/stable/ [9]
- https://fastapi.tiangolo.com/advanced/templates/#install-dependencies [10]
- https://fastapi.tiangolo.com/reference/templating/ [11]
- https://www.w3schools.com/ [12]
- https://fastapi.tiangolo.com/tutorial/static-files/ [13]
- https://docs.python.org/3/library/datetime.html#module-datetime [14]
- Silberschatz, A., Korth, H., & Sudarshan, S. *Database System Concepts.* *https://www.mpgcamb.com/wp-content/uploads/2024/12/Abraham-Silberschatz-Henry-F.-Korth-S.-Sudarshan-Database-System-Concepts-McGraw-Hill-Education-2019.pdf* [15]
- https://github.com/pyca/bcrypt/issues/1082 [16]
- https://pip.pypa.io/en/stable/cli/pip_cache/ [17]
- https://www.linkedin.com/pulse/experimenting-practicing-kali-linux-virtualbox-franklin-ukaegbu--mwgwe [18]
- https://stackoverflow.com/questions/75040507/how-to-access-fastapi-backend-from-a-different-machine-ip-on-the-same-local-netw [19]
- https://onsecurity.io/article/pentesting-postgresql-with-sql-injections/ [20]
- https://www.apisec.ai/blog/api-sql-injection-testing-payloads-guide [21]
- https://docs.python.org/3/library/sqlite3.html [22]
- https://darkmarc.substack.com/p/sqlmap-tool-identify-and-exploit [23]
- https://www.kali.org/tools/burpsuite/ [24]